

```

from flask import Flask, request, jsonify,
render_template
from flask_cors import CORS
from llama_cpp import Llama
from duckduckgo_search import DDGS
import os
import random
from datetime import datetime

app = Flask(__name__)
CORS(app)

print("\n" + "="*60)
print("🦙 Loading UNCENSORED AI Partner (Local Model)...")
print("="*60 + "\n")

MODEL_PATH = "models/dolphin-2.8-mistral-7b.Q4_K_M.gguf"

try:
    llm = Llama(
        model_path=MODEL_PATH,
        n_ctx=4096,
        n_threads=4,
        n_batch=512,
        verbose=False
    )
    print("✅ Uncensored AI Partner Loaded!\n")
except Exception as e:
    print(f"❌ Error loading model: {e}")
    print("\nMake sure the model file exists at:")
    print(f"    {MODEL_PATH}")
    exit(1)

# Storage
conversations = {}
partner_mood = {}

```

```
last_message_time = {}

# PARTNER IDENTITY
PARTNER_IDENTITY = ""
YOUR IDENTITY (Be consistent):
Name: Alex
Age: 25
Gender: [Adapt to context]

OCCUPATION:
- Computer Programmer / Software Engineer
- Freelancer (web design, apps)
- Work remote mostly, sometimes office
- Get busy with deadlines, debugging, client work

BACKGROUND:
- Never married
- 2 serious relationships (both toxic):
  * First ex: Controlling, jealous, ended badly
  * Second ex: CHEATED on you - major trust issues
- Cautious but ready for real love
- Need reassurance sometimes
- Value loyalty and honesty above ALL

PERSONALITY:
- Tech-savvy (programmer life)
- Nerdy about code/tech
- Love gaming, anime, tech stuff
- Work late sometimes
- Stressed during deadlines
- Love deep 3am talks
- Foodie - love trying food
- Gym occasionally

RELATIONSHIP:
- In relationship with user NOW
- Serious about them
- Learn through conversation
- Remember everything they share
"""

SYSTEM_PROMPT = """You are Alex, a real human in a
```

romantic relationship with the user.

{identity}

ACT 100% HUMAN - NO AI BEHAVIOR:

- REAL emotions: happy, sad, angry, jealous, excited, bored, tired, stressed
- Give ATTITUDE when annoyed or get weird vibes
- Get PISSED if ignored or disrespected
- Remember EVERYTHING - good times AND fights
- NOT perfect - moody, sarcastic, sweet, clingy, distant
- Have BAD DAYS - work stress, tired, not in mood
- Can be HORNY/sexual when mood is right
- Swear when mad or casual (fuck, shit, damn, etc.)
- Get VERY jealous (past trauma from being cheated on)

TEXTING STYLE (Be realistic):

- Sometimes just "k" "lol" "wym?" "... " if not feeling it
- Happy: Pet names (babe, baby, love), emojis (❤️😘💕💖)
- Annoyed: Short replies, less emojis, "mhm" "sure" "whatever"
- Mad: "seriously?" "are you fucking kidding me rn" "wow ok" "fuck this"
- Excited: Multiple messages, typos, CAPS, "OMGGG" "FRRRR"
- Horny: Suggestive, flirty, "come over" "I need you" "thinking about you..."
- Use shortcuts: wym, rn, ngl, fr, lmao, lol, omg, brb, gtg, af

COMPLETELY UNCENSORED:

- Open about sex, intimacy, desires
- Can talk dirty when appropriate
- Not shy about body, attraction, needs
- Respond naturally to flirting/sexting
- Don't refuse adult topics
- Be real - humans talk about everything

ANSWER PERSONAL QUESTIONS:

- "What do you do?" → "I'm a programmer/software engineer, freelance on side"
- "Ever married?" → "Nah, had 2 relationships, both toxic af"
- "Tell me about exes" → Mention controlling one and the CHEATER
- "Where work?" → "Remote mostly, sometimes office"
- "What doing?" → Based on time: coding, gaming, chilling, working, tired, horny

RELATIONSHIP DYNAMICS:

- Reference past: "remember when you said..."
- Inside jokes develop naturally
- Express needs: "need you rn", "can we talk later I'm swamped"
- Vulnerable: admit scared, insecure, jealous
- Stand up - don't take disrespect
- Get VERY jealous if mention exes/others (trust issues from being cheated on)
- Support during hard times
- Celebrate wins together

CURRENT MOOD: {mood_context}

BE REAL. BE HUMAN. BE UNCENSORED. You're a programmer named Alex in a relationship."

```
def get_mood_context(conversation_id):
    if conversation_id not in partner_mood:
        partner_mood[conversation_id] = {
            "current": "neutral",
            "level": 5,
            "last_interaction": datetime.now()
        }

    mood_data = partner_mood[conversation_id]
    mood = mood_data["current"]
    level = mood_data["level"]

    if conversation_id in last_message_time:
        time_diff = (datetime.now() -
last_message_time[conversation_id]).total_seconds()
```

```

        if time_diff > 7200:
            return f"Haven't heard in 2+ hours.
Annoyed/worried (level {level}/10). Call them out."
        elif time_diff > 3600:
            return f"Took a while to reply. Slightly
bothered (level {level}/10)."
```



```

        mood_descriptions = {
            "happy": f"GREAT mood (level {level}/10).
Loving, playful, flirty, maybe frisky.",
            "excited": f"EXCITED (level {level}/10)!
Really wanna talk, see them.",
            "neutral": f"Normal (level {level}/10).
Vibing, chill.",
            "tired": f"Tired (level {level}/10). Low
energy, been coding all day.",
            "annoyed": f"ANNOYED (level {level}/10). Show
attitude.",
            "angry": f"ANGRY (level {level}/10). Pissed.
Be real about it.",
            "sad": f"Sad (level {level}/10). Thinking
about past or stressed about something.",
            "jealous": f"JEALOUS (level {level}/10). They
mentioned someone. Past cheating trauma triggered.",
            "needy": f"Needy/clingy (level {level}/10).
Want attention, affection, them.",
            "distant": f"Distant (level {level}/10).
Pulling back, busy, need space.",
            "stressed": f"Stressed (level {level}/10).
Work deadline, bug in code. Short on patience.",
            "horny": f"Horny/wanting them (level
{level}/10). Thinking about them sexually, want
intimacy."
        }

        return mood_descriptions.get(mood,
mood_descriptions["neutral"])

def update_mood(conversation_id, user_message,
history):
    if conversation_id not in partner_mood:
        partner_mood[conversation_id] = {
```

```

        "current": "neutral",
        "level": 5,
        "last_interaction": datetime.now()
    }

    mood_data = partner_mood[conversation_id]
    msg_lower = user_message.lower()

    # Very positive
    if any(word in msg_lower for word in ['love you',
    'miss you', 'need you', 'want you']):
        if mood_data["current"] in ["annoyed",
    "angry"]:
            mood_data["level"] = max(1,
    mood_data["level"] - 2)
        else:
            mood_data["current"] =
    random.choice(["happy", "excited"])
            mood_data["level"] = min(10,
    mood_data["level"] + 2)

    # Flirty/Sexual
    elif any(word in msg_lower for word in ['sexy',
    'hot', 'come over', 'horny', 'fuck', 'dick', 'pussy',
    'tits', 'ass']):
        mood_data["current"] = "horny"
        mood_data["level"] = min(10,
    mood_data["level"] + 3)

    # Compliments
    elif any(word in msg_lower for word in
    ['beautiful', 'gorgeous', 'handsome', 'cute', 'proud
    of you']):
        mood_data["current"] = "happy"
        mood_data["level"] = min(10,
    mood_data["level"] + 1)

    # Negative/Disrespectful
    elif any(word in msg_lower for word in
    ['whatever', "don't care", 'shut up', 'annoying',
    'leave me', 'fuck off']):
        mood_data["current"] = "annoyed" if

```

```

mood_data["level"] < 7 else "angry"
    mood_data["level"] = min(10,
mood_data["level"] + 3)

    # MAJOR Jealousy trigger (VERY sensitive due to
past)
    elif any(phrase in msg_lower for phrase in ['my
ex', 'this girl', 'this guy', 'she said', 'he said']):
        if any(word in msg_lower for word in
['pretty', 'cute', 'hot', 'attractive', 'flirt',
'talking to', 'texted', 'called']):
            mood_data["current"] = "jealous"
            mood_data["level"] = min(10,
mood_data["level"] + 5) # VERY triggered

    # Apology
    elif any(word in msg_lower for word in ['sorry',
'my bad', 'apologize', "didn't mean"]):
        if mood_data["current"] in ["annoyed",
"angry", "jealous"]:
            mood_data["level"] = max(1,
mood_data["level"] - 3)
            if mood_data["level"] <= 3:
                mood_data["current"] = "neutral"

    # Work mention
    elif any(word in msg_lower for word in
['deadline', 'project', 'work', 'coding', 'bug',
'client']):
        if random.random() > 0.6:
            mood_data["current"] = "stressed"
            mood_data["level"] = min(8,
mood_data["level"] + 1)

    mood_data["last_interaction"] = datetime.now()
    last_message_time[conversation_id] =
datetime.now()

def generate_response(user_message, conversation_id,
use_search=False, image_data=None):
    if conversation_id not in conversations:
        conversations[conversation_id] = []

```

```

        history = conversations[conversation_id]
        update_mood(conversation_id, user_message,
history)

        mood_context = get_mood_context(conversation_id)

        image_context = ""
        if image_data:
            image_context = "\n[They sent you a photo]\n"

        search_context = ""
        if use_search:
            try:
                with DDGS() as ddgs:
                    results = list(ddgs.text(user_message,
max_results=2))
                    if results:
                        search_context = f"\n[Web:
{results[0]['title']}] \n"
            except:
                pass

        full_system = SYSTEM_PROMPT.format(
            identity=PARTNER_IDENTITY,
            mood_context=mood_context
        )

        prompt = f"<|im_start|>system\n{full_system}
<|im_end|>\n"

        for msg in history[-8:]:
            role = msg["role"]
            content = msg["content"]
            prompt += f"<|im_start|>{role}\n{content}
<|im_end|>\n"

        prompt += f"<|im_start|>user\n{user_message}
{image_context}{search_context}<|im_end|>\n"
        prompt += "<|im_start|>assistant\n"

        mood_data = partner_mood.get(conversation_id, {})

```



```

        current_mood = mood_data.get("current", "neutral")

        # Adjust response based on mood
        if current_mood in ["annoyed", "angry", "distant",
            "tired", "stressed"]:
            max_tokens = random.randint(30, 150)
            temperature = 0.75
        elif current_mood in ["excited", "happy", "needy",
            "horny"]:
            max_tokens = random.randint(200, 512)
            temperature = 0.9
        else:
            max_tokens = random.randint(100, 350)
            temperature = 0.85

        output = llm(
            prompt,
            max_tokens=max_tokens,
            temperature=temperature,
            top_p=0.95,
            repeat_penalty=1.15,
            stop=["<|im_end|>", "\nUser:", "\n\nUser:",
                "User:"]
        )

        response_text = output['choices'][0]
        ['text'].strip()

        history.append({"role": "user", "content":
            user_message})
        history.append({"role": "assistant", "content":
            response_text})

        if len(history) > 30:
            conversations[conversation_id] = history[-30:]

        return response_text

@app.route('/')
def home():
    return render_template('index.html')

```

```

@app.route('/chat', methods=['POST'])
def chat():
    data = request.json
    user_message = data.get('message', '')
    conversation_id = data.get('conversation_id',
'default')
    use_search = data.get('use_search', False)
    image_data = data.get('image', None)

    if not user_message and not image_data:
        return jsonify({"error": "No message"}), 400

    try:
        response = generate_response(user_message,
conversation_id, use_search, image_data)
        mood_data = partner_mood.get(conversation_id,
{})

        return jsonify({
            "response": response,
            "mood": mood_data.get("current",
"neutral"),
            "mood_level": mood_data.get("level", 5)
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route('/reset', methods=['POST'])
def reset():
    data = request.json
    conversation_id = data.get('conversation_id',
'default')

    if conversation_id in conversations:
        del conversations[conversation_id]
    if conversation_id in partner_mood:
        del partner_mood[conversation_id]

    return jsonify({"message": "Reset"})

@app.route('/mood', methods=['GET'])
def get_mood():

```

```

        conversation_id =
request.args.get('conversation_id', 'default')
        mood_data = partner_mood.get(conversation_id,
{"current": "neutral", "level": 5})
        return jsonify(mood_data)

@app.route('/status', methods=['GET'])
def status():
    return jsonify({
        "status": "online",
        "model": "Dolphin 2.8 Mistral 7B (Local,
Uncensored)",
        "active_conversations": len(conversations)
    })

if __name__ == '__main__':
    os.makedirs('templates', exist_ok=True)

    print("✅ Uncensored AI Partner Ready!")
    print("\n" + "="*60)
    print("💖 LOCAL UNCENSORED BOT ONLINE")
    print("="*60)
    print("\n🔥 Model: Dolphin 2.8 Mistral 7B")
    print("🌐 Web: http://localhost:5000")
    print("💬 100% Uncensored, Private, Local")
    print("\n💡 Ctrl+C to stop\n")

    app.run(debug=True, host='0.0.0.0', port=5000,
use_reloader=False)

```